

HOSTED BY

Wayflyer | Fin

— PARTICIPANT PACK · PRETTY FLY DATA —

HACKATHON

Build the future of
eCommerce →

03 WED
JUNE

• •

05 FRI
JUNE

ONE CHALLENGE → → → THREE DAYS TO BUILD

• LONDON · 2026

- WELCOME

You're holding 24 months of data for a fictional DTC business.

This pack contains everything you need to build something useful for Pretty Fly. A London streetwear brand selling premium tees, hoodies, sweatpants, caps and trainers since June 2024. Menswear at first, with a small womenswear line that launched in December 2025.

Your job over the next three days is to build them an AI-powered tool that helps them run their business better. The data is real-shaped: orders, inventory, ads, email, support, banking. And it reconciles, end to end.

- PICK YOUR PATH

There is no required direction.

Three rough framings below. Pick one, blend them, or invent your own. The best builds often live at the seam between lanes, but knowing where you're pointing keeps the work focused.

PATH 01

Customer-facing

Tools Pretty Fly might put in front of their customers: a smarter storefront, support agents, sizing assistants, post-purchase concierges, returns prediction, personalised journeys, loyalty.

PATH 02

Operator-focused

Tools the team uses internally: inventory and cash forecasting, ad efficiency triage, conversational P&L reporting, margin diagnostics, support automation, anomaly detection.

PATH 03

Surprise us

The dataset is rich. If you see a direction that doesn't fit either lane, take it. The strongest builds rest on a specific finding, not a generic idea.

- JUDGING CRITERIA

Execution

Does it work?

Business value

Would Pretty Fly pay for this?

Demo quality

Is your pitch compelling?

• WHAT'S IN THE BOX

21 files. One reconciled history.

Bank transactions tie to orders. Ad spend ties to bank. PO payments tie to supplier deliveries. Inventory movements tie to sales. If your numbers don't match, the bug is almost certainly yours, and the validator can prove it.

FOLDER STRUCTURE

pretty_fly_data_pack/		
└─ README.md	← this file	
└─ data/		
└─ products.csv		
└─ variants.csv		
└─ collections.csv		
└─ product_collections.csv		
└─ customers.csv		
└─ addresses.csv		
└─ orders.csv		
└─ line_items.csv		
└─ discount_codes.csv		
└─ refunds.csv		
└─ suppliers.csv		
└─ purchase_orders.csv		
└─ po_line_items.csv		
└─ inventory_movements.csv		
└─ google_ads_daily.csv		
└─ meta_ads_daily.csv		
└─ email_campaigns.csv		
└─ email_events.csv		
└─ support_tickets.csv		
└─ support_messages.json		
└─ bank_transactions.csv		
└─ validate.py	← reconciliatio	
└─ requirements.txt		

ROW COUNTS

FILE	ROWS
CATALOGUE	
products.csv	62
variants.csv	645
collections.csv	9
product_collections.csv	62
CUSTOMERS & ORDERS	
customers.csv	22,440
addresses.csv	22,440
orders.csv	49,793
line_items.csv	69,956
discount_codes.csv	8
refunds.csv	5,843
SUPPLY & INVENTORY	
suppliers.csv	5
purchase_orders.csv	21
po_line_items.csv	645
inventory_movements.csv	76,444
MARKETING & SUPPORT	
google_ads_daily.csv	5,110
meta_ads_daily.csv	3,102
email_campaigns.csv	6
email_events.csv	11,368
support_tickets.csv	1,204
support_messages.json	1,204
BANKING	
bank_transactions.csv	560

- QUICK START

Join through orders, line items, variants, products.

A typical join, and a monthly revenue rollup. Most paths through the data start here.

```
import pandas as pd

orders      = pd.read_csv("data/orders.csv", parse_dates=["created_at"])
line_items  = pd.read_csv("data/line_items.csv")
products    = pd.read_csv("data/products.csv")
variants    = pd.read_csv("data/variants.csv")

# A typical join: orders → line items → variants → products
sold = (line_items
        .merge(orders[["order_id", "created_at", "customer_id"]], on="order_id")
        .merge(variants[["variant_id", "product_id", "sku"]], on="variant_id")
        .merge(products[["product_id", "title", "product_type", "gender_segment"]], on="product_id"))

# Monthly revenue
sold["month"] = sold["created_at"].dt.to_period("M")
sold.groupby("month").apply(lambda x: (x["price"] * x["quantity"]).sum())
```

- THE DATASET AT A GLANCE

How each table connects.

Catalogue products · variants · collections · product_collections

Products are the abstract item ("Heritage Hoodie"). Variants are the actual sellable units. A hoodie in Washed Black, size M, has its own SKU and inventory level. Products belong to one or more collections (Core, SS25 Drop, Womens, etc.).

Customers customers · addresses

~22,400 unique customers. Each row has an acquisition source (UTM of their first order), a default country, and a derived `gender_segment_affinity` based on purchase history. Useful for cohort reasoning.

Orders orders · line_items · discount_codes · refunds

~50,000 orders across 24 months. Each has subtotals, discounts, shipping, VAT (UK inclusive; export orders don't), and UTM attribution. Refunds link back with a reason. Discount codes that appear on orders are real Pretty Fly codes.

Inventory & supply suppliers · purchase_orders · po_line_items · inventory_movements

Pretty Fly buys from suppliers in Portugal, Italy, and Turkey. Each PO has a deposit, a balance, and a delivery date. Every receipt, sale, and return creates a movement. Sum them per variant to get current stock. The gap between paying suppliers and earning back revenue is the brand's cash conversion cycle.

Advertising google_ads_daily · meta_ads_daily

Daily spend, clicks, impressions, conversions and conversion value across both platforms over 24 months. Campaign names match the `utm_campaign` field on orders, so you can tie marketing spend directly to attributed revenue. Standard caveat: ad platforms overclaim vs. what Shopify actually attributes.

Email email_campaigns · email_events

Klaviyo-style. Campaigns are one-offs (a drop announcement); flows are automated (welcome series, abandoned cart). Events show every send, open, click and conversion at the customer level. Email-attributed revenue is fully reconcilable.

Support support_tickets · support_messages

~1,200 tickets across the 24 months. Channels are email, live chat and Instagram DM. Messages are realistic conversations between customers, Pretty Fly's human agents and their existing bot triage. ~40% of tickets are bot-resolved today; the rest go to humans. Each ticket links to a customer, an order, or a product.

Banking bank_transactions

Every penny in and out of the business over 24 months. Counterparties have realistic, messy descriptions (`SHOPIFY* 8829 PAYOUT REF#82910` , `FB*ADS9921XK MENLO PARK`). The `counterparty` field is fully populated with the clean entity behind each transaction. `raw_category` is coarse (`SUPPLIER` , `PAYROLL` , `SAAS` , `RENT` , `MARKETING` , `SHIPPING` , `FULFILMENT` , `PAYOUT` , `REFUND` , `OTHER`), useful for a fast first-pass P&L. `category` is deliberately blank; categorising finely is a build direction in itself.

• BUILD INSPIRATION**Questions Pretty Fly's team can't answer cleanly today.**

We're not going to tell you what to build. But the data should let you tackle things like:

- Where is cash actually getting tied up, and what would it cost to fix?
- Which marketing pounds are working hardest, and which should they pull tomorrow?
- Are there customer segments quietly outperforming, or quietly underperforming, that they should treat differently?
- Which support tickets could a bot resolve end-to-end if it had access to order and product data?
- Which products would they reorder right now if they could see clearly? Which would they walk away from?
- What does their P&L actually look like this month, and how would they know without an accountant?
- If you were Pretty Fly's CFO, what would worry you in this data? If you were their head of growth, what would excite you?

- BUILDING P&LS FROM THIS DATA

A few pointers if that's your direction.

P&L tools are one of the most natural build paths in this dataset. Three lines need more than just bank transactions:

COGS is recognised when units are sold, not when suppliers are paid.

To compute it cleanly, multiply units sold (from `line_items`) by their landed cost per unit (from `po_line_items`).

Shopify payment processing fees are embedded inside payout amounts.

Payouts equal gross orders minus fees and refunds for that batch. Derive the fee by comparing payout amounts to the orders included.

Returns affect both the refunds table and bank_transactions.

SHOPIFY REFUND BATCH lines on banking; `refunds` is more granular. The simplest first-pass P&L groups bank transactions by `raw_category` and adjusts for the items above. The more accurate version joins through orders, line items, PO line items, and refunds. Both are reasonable scopes; pick what fits your time.

- VERIFYING YOUR NUMBERS

20 reconciliation rules. Run them whenever you're unsure.

If you're ever unsure whether a number you've computed is right, or whether a discrepancy is your code or the data, run the validator.

```
$ pip install -r requirements.txt
$ python validate.py data/
```

It checks order subtotals, customer aggregates, inventory balances, ad spend vs. bank, refund timing, UTM consistency across tables, and a dozen other things. All rules should pass on the dataset you've been given. If any fail, tell us, but they shouldn't. The same script is a useful reference for how tables relate; read it if you're stuck.

• WHAT'S NOT IN HERE

To save you searching for it.

- **No real customer data.** Every name, email and address is synthetic, like `firstname.lastname@example-fake.com`.
- **No live API.** This is a snapshot, not a connection. If you want to simulate one, build it.
- **No payment processor data** beyond what Shopify exposes. No Stripe-level fraud signals, no card-level info.
- **No web analytics.** No GA, no session data, no page views. Assume or generate if you need it.
- **No images.** Products are described, not pictured. Generate or stub as needed.

• TIPS FOR A STRONG BUILD

What separates the strongest builds.

- **Spend the first few hours exploring.** The strongest builds rest on a specific finding, not a generic idea. Day 1 is investigation; Day 2 is building on what you found.
- **Don't try to use every table.** A focused build on three tables beats a broad build on twelve.
- **Demos beat slides.** A working interface speaks louder than mockups every time.
- **AI is a tool, not the product.** Judges aren't impressed by "we used the latest model". They're impressed by what you built with it.
- **Have fun.** Fictional brand, real-shaped data. Ship something you'd be proud to demo on Friday.

• PRACTICAL REMINDERS

Timestamps

UTC throughout. Convert to Europe/London for local-time patterns (Black Friday spikes, drop launches).

Currency

GBP. Some supplier invoices are in EUR/USD with conversion noted; everything reports in GBP.

VAT

Inclusive in UK retail prices. A £50 tee is £50 to a UK customer (£8.33 VAT inside); a US customer pays £41.67 net.

Window

1 Jun 2024 → 31 May 2026.
"Today" in this world is the morning of 1 Jun 2026.

Womenswear

Launched Dec 2025, six months before the dataset ends. Pre-launch the brand was menswear-only.

Attribution key

`utm_campaign` is the master key. It threads through orders, ads, and email. Discount codes are case-sensitive, as on Shopify.

GETTING HELP · TOOLS DOWN FRI 7PM

Wayflyer folks are on-site all three days. Chat to us in person or ask a question in the Discord channel.

discord.gg/gt3MUwRZ7